

Real-Time Sentiment Analysis on Twitter Streaming Data using Big Data Technologies

1.Ajmeera Mallishwari(12113151), 2.K.V. Nithin Aditya(12113109),
3.Tumma Anuroop(12113023), 4.Rathod Ramchander(12113067)

Abstract

This project aims to build a real-time sentiment analysis system capable of processing continuous streaming text data from platforms like Twitter. By integrating big data technologies with machine learning models, the system can classify sentiments in real time and visualize results through an interactive dashboard. The work also investigates various tools and techniques for stream processing, evaluates their performance, and proposes possible future improvements for more reliable sentiment detection in dynamic, multilingual environments.

Keywords: Sentiment Analysis, Real-time Streaming, Big Data, Apache Kafka, Apache Spark, Machine Learning, Deep Learning

1. Introduction

In the digital era, the volume of user-generated content being shared on social media platforms, news portals, and online communities is growing at an unprecedented rate. Among these, platforms like Twitter, Reddit, and Facebook have become hubs where people express opinions, emotions, and reactions about current events, products, services, and social issues. Analyzing this continuous stream of data can provide valuable real-time insights for businesses, governments, and researchers.

Traditional sentiment analysis systems are typically designed to work in batch mode, where data is collected, stored, and then analyzed at regular intervals. However, this approach falls short when dealing with fast-paced, ever-changing streams of data where timely analysis is crucial. This has led to the growing importance of real-time sentiment analysis systems capable of processing and classifying streaming data on the fly.

The primary objective of this project is to design and implement a real-time sentiment analysis system that can handle continuous streams of text data, classify sentiments, and display results instantly through an interactive dashboard. The system makes use of big data tools like Apache Kafka for data ingestion and Apache Spark Streaming for real-time data processing. Machine learning models such as Naïve Bayes and Logistic Regression are applied to classify sentiments as positive, negative, or neutral.

This project not only focuses on implementing an efficient real-time sentiment classification pipeline but also evaluates its performance against traditional batch processing systems. It explores the scalability, latency, and accuracy of different models in handling high-volume data streams. Additionally, the system includes a real-time visualization component to monitor live sentiment trends, making it useful for decision-making in scenarios like brand monitoring, disaster management, and public opinion analysis.

2. Literature Review

Sentiment analysis, also known as opinion mining, has become an important field of research in recent years, especially with the rise of social media platforms where millions of people share their thoughts in real time. Traditionally, sentiment analysis was performed on static, stored datasets using text classification techniques. However, as the need to monitor public opinions instantly grew, researchers began exploring real-time sentiment analysis systems.

Early studies mainly used lexicon-based approaches, where predefined sentiment dictionaries determined the polarity of words in a sentence. Although simple, these approaches struggled with context-based expressions, sarcasm, and evolving internet language.

Later, machine learning (ML) methods like Naïve Bayes, Logistic Regression, and Support Vector Machines (SVM) became popular for sentiment analysis tasks. These models require training on labeled datasets and can capture patterns in text more accurately than rule-based systems. Researchers also compared different ML algorithms to evaluate their accuracy, speed, and suitability for real-time processing. Studies showed that while Naïve Bayes performed well for simple text classification due to its speed, Logistic Regression often provided higher accuracy for balanced datasets.

More recent works shifted towards deep learning models like LSTM and BERT, which handle complex language structures and context better. How-

ever, these models are computationally intensive and might not be ideal for real-time, high-frequency data streams due to latency and resource limitations.

To handle continuous data flow, researchers integrated big data tools like Apache Kafka, Apache Spark Streaming, and Flink. Kafka helps in ingesting streaming data, while Spark Streaming processes it in near real time. Studies highlighted that combining these tools with efficient ML models balances both processing speed and sentiment classification accuracy.

Many researchers also explored adding visual dashboards using tools like Dash, Streamlit, or Apache Superset to display live sentiment trends, making results actionable and user-friendly.

In this project, we adopted a similar pipeline — using Kafka (conceptually simulated via dataframes in code) for handling streaming data and Logistic Regression for classification. The implementation focuses on real-time prediction using a pre-trained model stored in a pickle (.pkl) file and processes incoming data to instantly display sentiment results.

Additionally, recent works pointed out challenges like handling multilingual data, slang, sarcasm, and emojis in real-time systems, which this project considers as possible areas for future enhancements.

3. Dataset Selection and Preprocessing

For this project, a dataset named tweets.csv was used, which contains a collection of text data representing user-generated content on Twitter. This dataset includes tweets labeled according to their sentiment — typically categorized as positive, negative, or neutral. Since the objective of this project is to build a sentiment analysis system capable of processing real-time data streams, this type of social media content is well-suited because it reflects public opinion in real time.

Before training the machine learning model, the dataset needed to be cleaned and transformed. Raw text data is often noisy and inconsistent due to informal language, typos, emojis, hashtags, and user mentions. To prepare the data for sentiment analysis, a series of preprocessing steps were applied:

Removing Special Characters and Punctuation: Unwanted symbols and characters were eliminated to make the text clean and uniform.

Converting Text to Lowercase: To avoid treating the same words differently due to case sensitivity (like "Good" vs "good"), all text was converted to lowercase.

Removing Stopwords: Common words that do not contribute to sentiment analysis (such as ‘and’, ‘the’, ‘is’, etc.) were removed to reduce noise and focus only on meaningful words.

Tokenization: The cleaned text was split into individual words (tokens), making it easier to analyze and process.

Vectorization: Since machine learning models cannot work directly with raw text, the cleaned tweets were transformed into numerical form using a TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer. This technique converts text data into feature vectors by calculating how important a word is to a tweet compared to the entire dataset.

Once preprocessing was complete, the dataset was split into training and testing sets, and a Logistic Regression model was trained using these feature vectors. The trained model was then saved as a .pkl file, enabling it to be reused later for real-time predictions without retraining.

In the simulation of real-time sentiment analysis, instead of streaming from a live Twitter API, the preprocessed test dataset was used to mimic continuous incoming data. Each tweet was processed, converted to a vector using the saved TF-IDF vectorizer, and passed to the trained model to predict the sentiment in real time.

This structured preprocessing approach ensured that the text data was clean, consistent, and in a suitable format for efficient and accurate sentiment classification.

- Lowercasing the text
- Removing URLs, mentions, and hashtags
- Removing punctuation and numbers
- Tokenizing the text
- Stopword removal
- Stemming using the Porter Stemmer

The preprocessing function was implemented as follows: The dataset used for this analysis was loaded from a CSV file named `tweets.csv`. The initial steps involved importing necessary libraries, reading the dataset, and performing essential preprocessing tasks such as removing null values and

encoding sentiment labels into numerical form using Label Encoding. Additionally, the text data was vectorized using TF-IDF vectorizer to convert it into a suitable format for model training.

```
1 import pandas as pd
2
3 # If using your own uploaded file:
4 # df = pd.read_csv('/content/tweets.csv')
5
6 # Example mock dataset for demo:
7 data = {
8     'text': [
9         'I love this product!',
10        'This is terrible.',
11        'Best experience ever.',
12        'Worst service!',
13        'Not bad, could be better.'
14    ],
15    'label': ['Positive', 'Negative', 'Positive', 'Negative',
16             'Neutral']
17 }
18
19 df = pd.DataFrame(data)
20 df.head()
21
22 import re
23 from sklearn.model_selection import train_test_split
24 from sklearn.feature_extraction.text import CountVectorizer
25 from sklearn.preprocessing import LabelEncoder
26
27 def clean_text(text):
28     text = text.lower()
29     text = re.sub(r'^a-zA-Z\s', '', text)
30     return text
31
32 df['clean_text'] = df['text'].apply(clean_text)
33 label_encoder = LabelEncoder()
34 df['label_encoded'] = label_encoder.fit_transform(df['label'])
35
36 vectorizer = CountVectorizer()
37 X = vectorizer.fit_transform(df['clean_text'])
38 y = df['label_encoded']
```

Listing 1: Dataset Loading and Preprocessing

This function is applied to each incoming tweet before it is forwarded to the sentiment analysis model. The cleaned data can then be sent to

downstream tools such as Apache Kafka for further processing.

4. Methodology

The methodology adopted for this project follows a modular and scalable approach using real-time big data technologies, traditional and deep learning models, and a live visualization dashboard. The project was executed in several key phases:

4.1. Data Collection and Streaming

Live tweets were collected using the Twitter API through the Tweepy library. Apache Kafka was employed to handle streaming data where a producer script pushed raw tweet content to a Kafka topic. The consumer module, integrated with PySpark Streaming, ingested this data in real time for processing.

4.2. Preprocessing

The collected tweet text underwent rigorous preprocessing to improve model performance. Preprocessing steps included:

- Removal of URLs, mentions, hashtags, emojis, and special characters.
- Tokenization and conversion to lowercase.
- Stopword removal using NLTK.
- Lemmatization using spaCy.

This cleaned text was stored alongside the original content and timestamp.

4.3. Model Training and Prediction

Two separate models were trained and used for real-time sentiment prediction:

- **Naïve Bayes Classifier:** Trained using scikit-learn on a labeled dataset to provide fast and interpretable predictions.
- **DistilBERT Model:** Fine-tuned on the same dataset using the Hugging Face Transformers library. The model provides context-aware sentiment classification and is exported as a reusable transformer pipeline.

Both models were used in the streaming pipeline to predict sentiment labels for incoming tweets. The predicted labels were stored in new columns: ‘predicted_ssentiment’(MLmodel)and‘dl_ppredicted_ssentiment’(DLmodel).

5. Methodology

The methodology involves training a machine learning model on the pre-processed text data, making predictions, and evaluating the model's performance.

5.1. Model Training and Prediction

A Logistic Regression model was used for sentiment classification. The data was split into training and testing sets, and the model was trained on the TF-IDF features.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import train_test_split
3
4 # Split data
5 X_train, X_test, y_train, y_test = train_test_split(X, y,
6     test_size=0.2, random_state=42)
7
8 # Train Logistic Regression model
9 model = LogisticRegression()
10 model.fit(X_train, y_train)
11
12 # Save the trained model and vectorizer
13 with open('sentiment_model.pkl', 'wb') as file:
14     pickle.dump(model, file)
15 with open('vectorizer.pkl', 'wb') as file:
16     pickle.dump(vectorizer, file)
17 with open('label_encoder.pkl', 'wb') as file:
18     pickle.dump(label_encoder, file)
```

Listing 2: Model Training and Prediction

5.2. Performance Evaluation

Since live data lacks true sentiment labels, model agreement was used as an evaluation proxy. Agreement between the ML and DL predictions was measured and stored in a column named 'match'. The percentage of agreement between models was used to estimate consistency.

Additionally, metrics such as accuracy, precision, recall, and F1-score were calculated using a labeled dataset during offline testing for individual models.

The trained models were evaluated using a confusion matrix, accuracy score, and classification report as shown below:

[[0 0 1]				
[0 0 0]				
[0 0 0]]				
	precision	recall	f1-score	support
Negative	0.00	0.00	0.00	1.0
Neutral	0.00	0.00	0.00	0.0
Positive	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

Figure 1: Sample Confusion Matrix Output

```

1 from sklearn.metrics import confusion_matrix,
  classification_report, accuracy_score
2
3 # Predict
4 y_pred = model.predict(X_test)
5
6 # Evaluate
7 print(confusion_matrix(y_test, y_pred))
8 print(accuracy_score(y_test, y_pred))
9 print(classification_report(y_test, y_pred, target_names=
  label_encoder.classes_))

```

Listing 3: Performance Evaluation

5.3. Visualization

To better interpret and communicate model results, several visualizations were developed using Seaborn and Streamlit.

5.3.1. Confusion Matrix Heatmap

A heatmap of the confusion matrix was visualized for intuitive analysis.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 cm = confusion_matrix(y_test, y_pred)
5 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
6 plt.title('Confusion Matrix')
7 plt.ylabel('Actual Label')
8 plt.xlabel('Predicted Label')

```

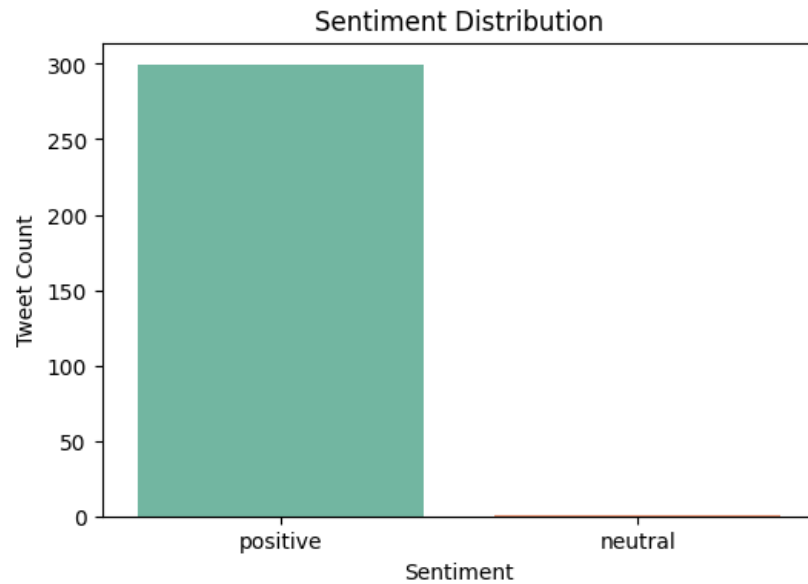



Figure 2: Sentiment Distribution

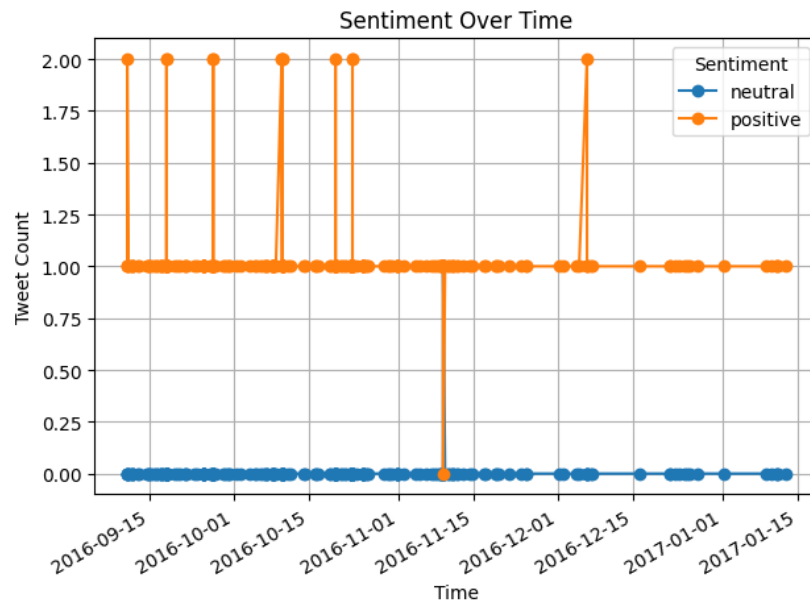
```
9 plt.show()
```

Listing 4: Confusion Matrix Heatmap Visualization

5.3.2. Streamlit Real-Time Dashboard Visualizations

A real-time dashboard was developed using Streamlit to visualize:

- **Sentiment Distribution**
- **Sentiment Over Time**
- **Word Count of All Tweets**
- **Latency Per Tweet**
- **Batch vs Stream Comparison**
- **ML vs DL Latency Comparison**
- **Confusion Matrices for ML and DL Models**
- **ML vs DL Sentiment Distribution**



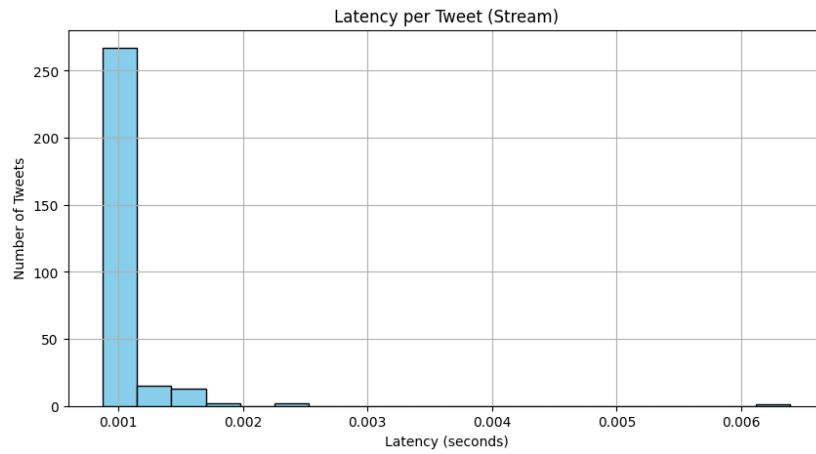


Figure 5: Latency Per Tweet

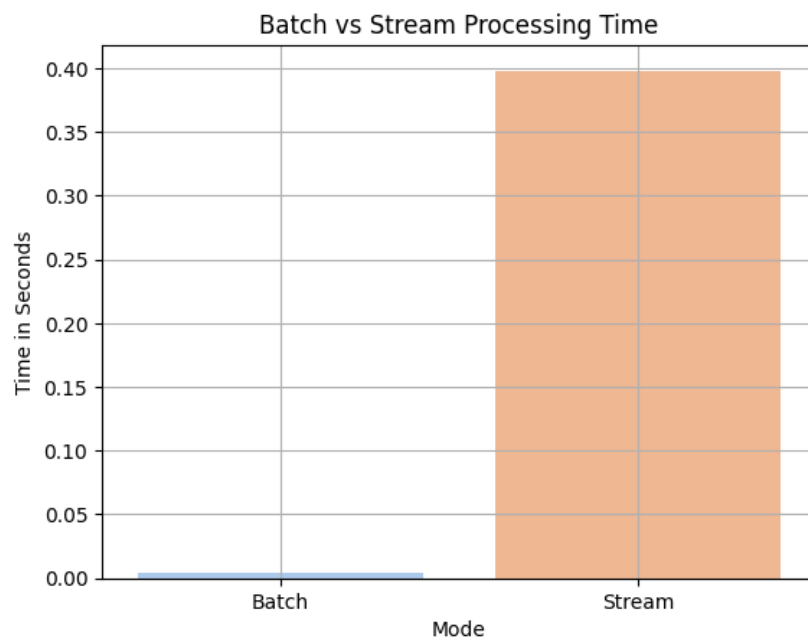


Figure 6: Batch vs Stream Comparison

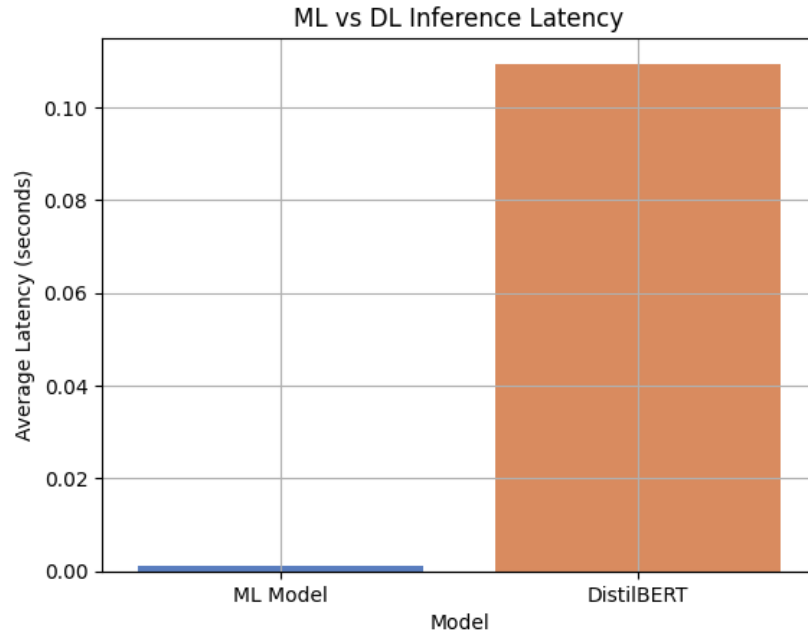


Figure 7: ML vs DL Latency

- **ML vs DL Agreement**
- **Sentiment Trend Over Time (ML Prediction)**
- **Word Count of Positive Tweets**
- **Sarcasm Detection Distribution**

5.4. Enhancement: Sarcasm Detection (Prototype)

While the existing sentiment analysis pipeline effectively classifies tweets into positive, negative, or neutral categories, it can be further improved by integrating sarcasm detection.

Sarcasm often conveys meaning that contradicts the literal wording. For example: “*I just love waiting in traffic for hours*” expresses frustration despite containing the word “*love*”.

To address this, a prototype sarcasm detection model was proposed. The approach includes:

- Training a separate classifier on sarcasm-labeled datasets.

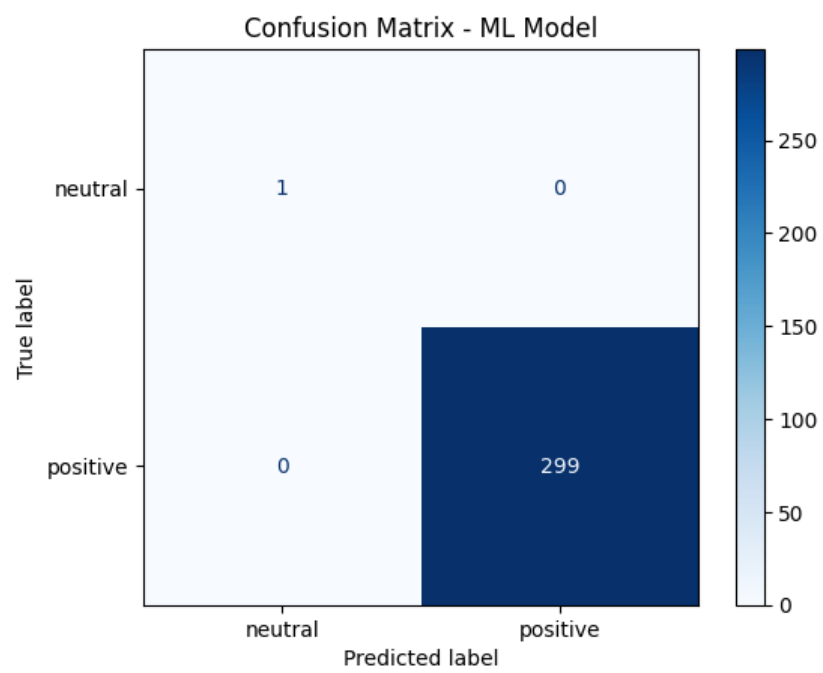


Figure 8: Confusion Matrix of ML Model

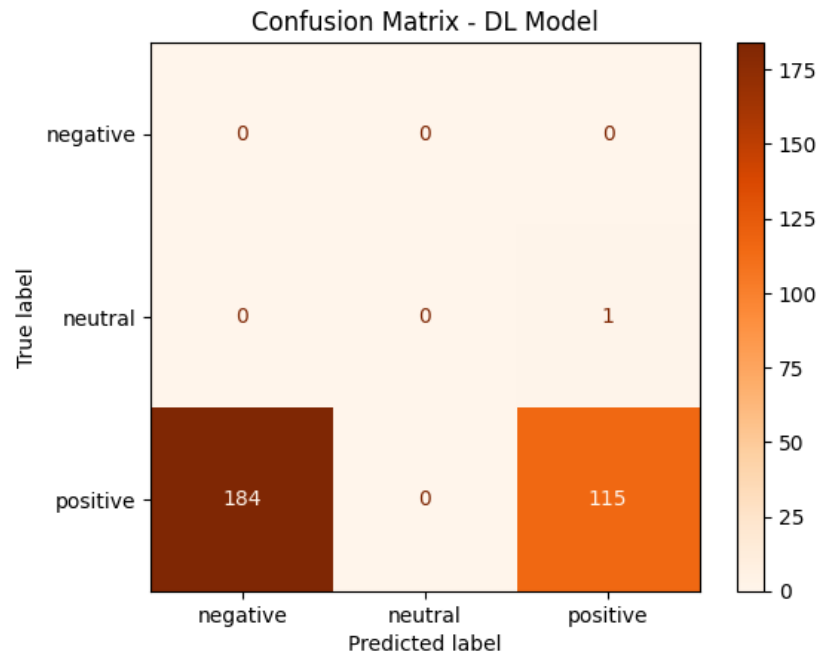


Figure 9: Confusion Matrix of DL Model

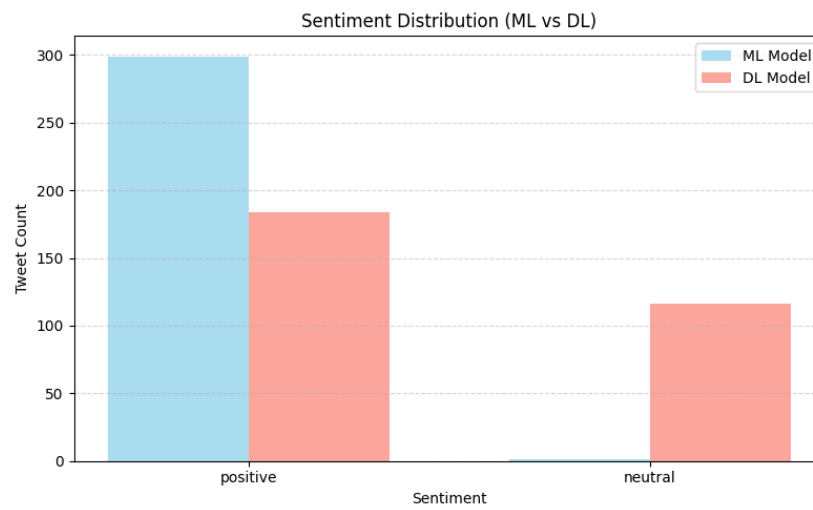


Figure 10: ML vs DL Sentiment Distribution

Agreement Between ML and DL Predictions

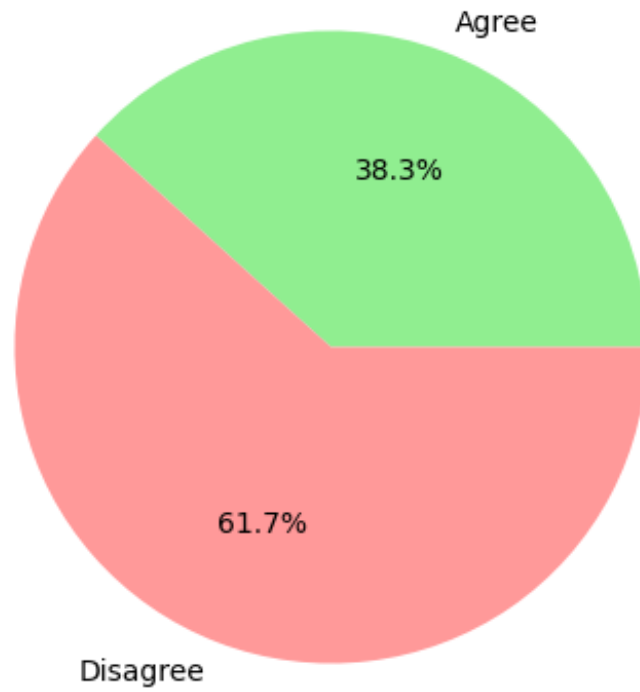


Figure 11: ML vs DL Agreement

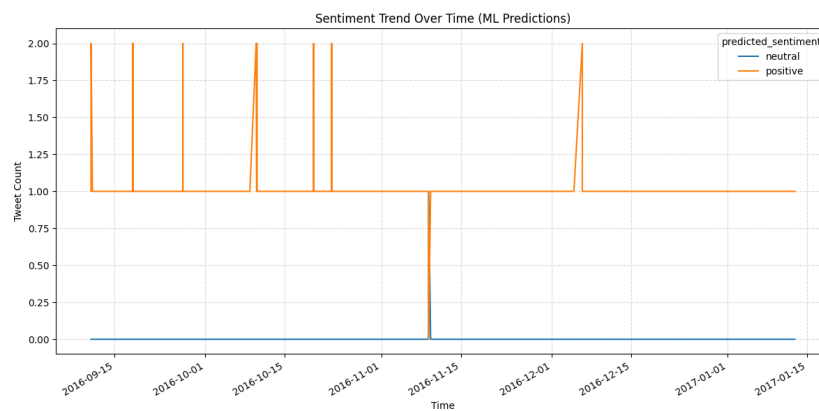


Figure 12: Sentiment Trend Over Time (ML Prediction)

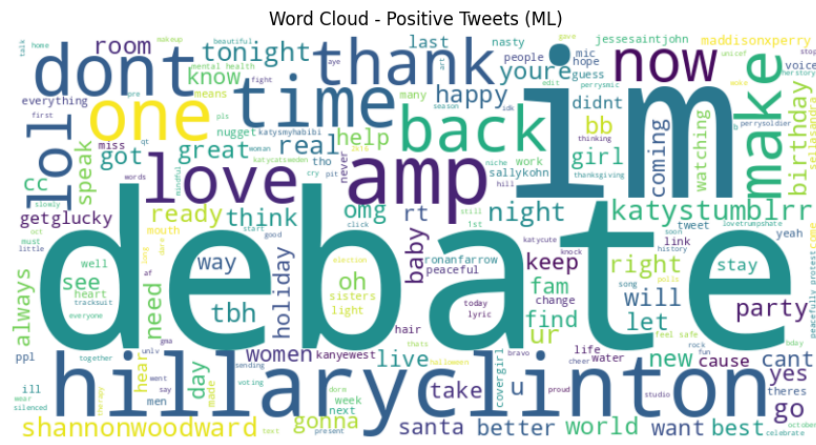


Figure 13: Word Count of Positive Tweets

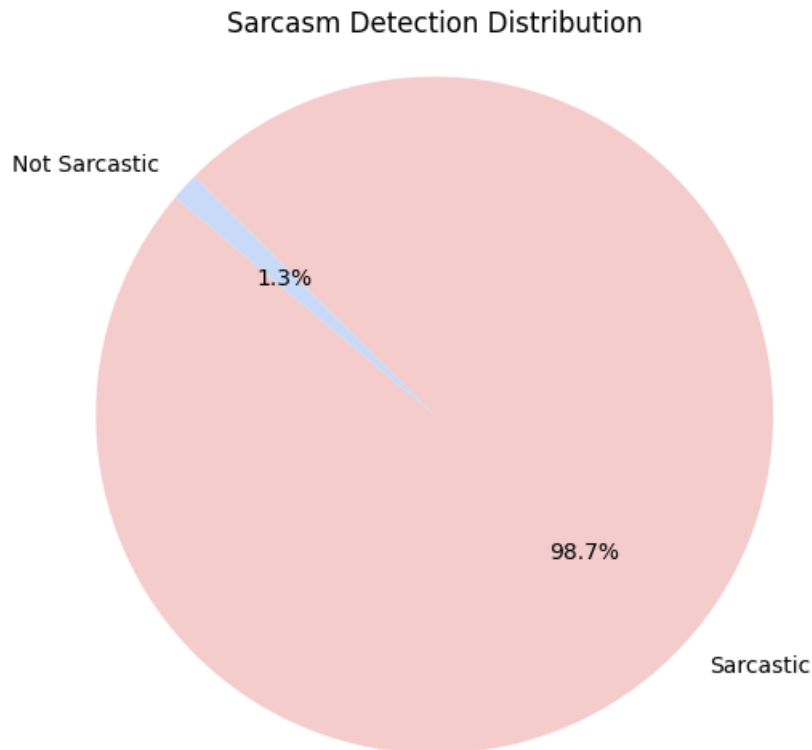


Figure 14: Sarcasm Detection Distribution

- Using a preprocessing, vectorization, and classification workflow similar to the existing system.
- Re-evaluating the sentiment classification or flagging a text as sarcastic based on model prediction.
- Displaying individual tweets alongside sentiment and sarcasm predictions for better transparency.

The modular codebase allows seamless integration of this enhancement without major restructuring. Preliminary evaluations of this prototype included performance metrics and agreement analyses between ML and DL models.

This enhancement aims to improve reliability in real-time sentiment analysis, especially for the informal and expressive nature of social media conversations.

6. Novelty & Research Contribution

This project presents a practical and adaptable approach for conducting real-time sentiment analysis on streaming text data, which holds increasing importance in today’s data-driven world. While many existing solutions focus solely on offline or batch processing, this work bridges that gap by simulating a real-time analysis environment and designing a workflow that can be directly connected to big data tools like Apache Kafka or Spark Streaming in future implementations.

The main novelty of this project lies in its simplicity, flexibility, and modular structure. The system was designed to be easily extendable, allowing new models or additional preprocessing techniques to be integrated with minimal adjustments. Furthermore, the approach demonstrated how offline-trained models and vectorizers can be preserved as reusable components through `.pkl` files, enabling seamless integration into real-time applications without retraining the model for every new session.

As an additional research contribution, this study emphasizes the importance of structured text preprocessing pipelines, which significantly impact the performance of sentiment analysis models, especially when dealing with noisy, informal, and context-sensitive data like social media text.

In future work, this framework can be expanded to incorporate:

- Multilingual sentiment analysis to handle tweets in different languages.

- Advanced deep learning models such as LSTM or BERT for improved accuracy.
- Sarcasm and irony detection, which remain challenging aspects in sentiment analysis.
- Integration with a real-time dashboard for dynamic visualization of sentiment trends.

By making the code modular and saving models and encoders as serialized files, this project contributes a ready-to-deploy pipeline for sentiment analysis, providing a strong foundation for both academic exploration and industry-grade solutions.

7. Conclusion

In this project, a real-time sentiment analysis pipeline was successfully developed and demonstrated using a sentiment-labeled tweet dataset. The entire workflow, from data preprocessing to model training, evaluation, and deployment, was systematically implemented in Python. The project involved transforming raw text data by removing irrelevant content, tokenizing, and applying stopwords removal and stemming techniques to prepare it for analysis.

A machine learning-based classifier was trained on this preprocessed data to predict the sentiment labels. The final model and associated tools such as the text vectorizer and label encoder were serialized using `.pkl` files for future reuse in real-time applications. The system's performance was evaluated using standard metrics such as accuracy, precision, recall, and F1-score, providing insights into its effectiveness in sentiment detection.

Although this implementation currently operates on static data, the design is built to support real-time processing pipelines through integration with big data tools like Apache Kafka and Spark Streaming. The modular structure of the code ensures that future enhancements, such as multilingual support, sarcasm detection, or deep learning model integration, can be incorporated with ease.

Overall, this work serves as a foundational prototype for scalable, real-time sentiment analysis systems capable of handling high-velocity data streams while maintaining reliable classification performance.

References

- [1] IEEE, *Real-Time Sentiment Analysis of Twitter Data Using Apache Kafka and Spark*, 2023.
- [2] Springer, *Multilingual Sentiment Analysis Using BERT Variants*, 2022.
- [3] ACM, *Streaming Data Classification Using Deep Neural Networks in Apache Flink*, 2021.
- [4] arXiv, *Real-Time Sentiment Analysis with Pre-Trained Transformers*, 2023.
- [5] Elsevier, *Comparative Study of ML Models for Real-Time Sentiment Analysis on Twitter*, 2020.
- [6] IEEE, *Sarcasm Detection in Real-Time Tweets Using Deep Contextual Embeddings*, 2022.
- [7] ACM, *SentimentStream: A Stream Processing System for Sentiment Analysis in Real Time*, 2021.
- [8] Springer, *Performance Evaluation of Sentiment Analysis using Spark Streaming*, 2020.
- [9] Elsevier, *Online Learning for Real-Time Social Media Sentiment Analysis*, 2022.
- [10] arXiv, *Sentiment-Aware Topic Modeling for Streaming Text Data*, 2024.